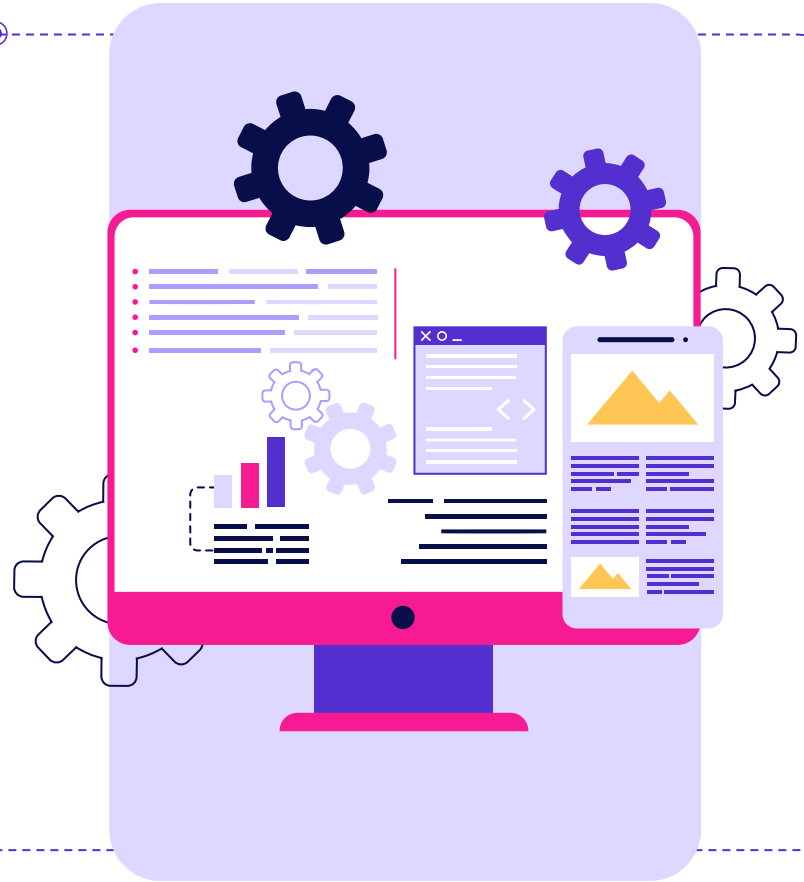


Johnson Yep's PORTFOLIO

Mechatronics Engineering @ uWaterloo

Scroll or use arrow keys to navigate



What?

I developed a real-time computer vision system to automate rack detection and decision-making for autonomous mobile robots (AMRs) in a manufacturing environment during my co-op term at Toyota Boshoku Canada.

Using **Python** and **OpenCV**, the system processes a live RTSP camera feed and applies homography and distortion correction to generate a consistent bird's-eye view of the workspace.

On top of this, I built a flexible detection pipeline using HSV colour segmentation and configurable regions to accurately determine rack occupancy in real time.

Impacts

This project replaced manual input with a fully automated system, significantly improving efficiency and consistency on the plant floor.

It enabled AMRs to make real-time decisions based on actual rack availability, helping the system reliably move over **100** racks per day while reducing operator workload and minimizing delays.

How?

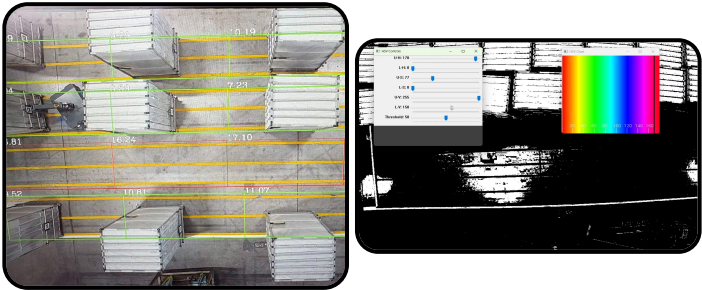
The system integrates directly with an Allen-Bradley PLC over Ethernet/IP, where detection results are converted into structured boolean arrays and used to trigger robot actions.

I designed an interactive calibration interface with live tuning for color thresholds, detection sensitivity, and geometric transformations, along with save/load functionality for rapid deployment.

To ensure performance and reliability, I implemented a multithreaded camera pipeline to eliminate latency and maintain stable real-time operation in a production setting.

AMR Computer Vision

Python • OpenCV • Real-Time Computer Vision
• PLC Integration • Multithreading • RTSP Streaming



What?

I developed human-machine interface (HMI) displays for multiple machines and robotic systems across Toyota Boshoku's production plant to improve monitoring, diagnostics, and operator control.

Using FactoryTalk **View Studio** and **Studio 5000**, I created interfaces that display real-time servo speed and torque data for sonic weld robots during clip installation. I also built tracking systems to monitor the total number of screws and clips installed, enabling maintenance teams to anticipate tool wear and schedule replacements proactively.

Additionally, I designed an interface that allows operators to selectively bypass vision-based defect detection for specific screws when needed, improving operational flexibility.

How?

The system integrates with **Allen-Bradley PLCs**, where I wrote **ladder logic** to capture, store, and manage real-time production and diagnostic data.

This data is communicated to the HMI through structured tags and arrays, enabling dynamic visualization and user interaction. I designed intuitive layouts and controls within FactoryTalk View to ensure operators and maintenance staff can easily interpret system status and make informed decisions.

Impacts

The HMIs improved visibility into robot performance, enabling faster troubleshooting and better decision-making. They enabled proactive maintenance through tool usage tracking, reducing unexpected downtime. They also increased operator flexibility, minimizing unnecessary production stoppages.

HMI Development

FactoryTalk View Studio • FactoryTalk Studio 5000 • Allen Bradley PLCs • Ladder Logic



What? I developed a real-time computer vision-based guitar interaction system that uses a webcam feed to detect hand and finger movements and translate them into musical input. Built with Python, OpenCV, and MediaPipe, the program tracks both hands simultaneously: the left hand selects chords based on finger positions, while the right hand triggers individual notes. I also added visual overlays such as chord diagrams and fingertip indicators to make the system more intuitive and interactive for the user.	How? I implemented custom hand-tracking logic to interpret finger states by comparing landmark positions and determining whether specific fingers were raised or pressed. I mapped these detected gestures to chord and note audio files using Pygame, then designed an event-based playback system that only triggered sounds on new finger presses, preventing repeated playback from continuous input. To keep the experience responsive, I optimized the program for real-time performance while continuously updating detections from the live camera feed.
Impacts This project sparked my interest in real-time computer vision by showing me how live camera input from a simple web cam could be transformed into an interactive, low-latency system. It introduced me to the challenge of making vision-based applications both accurate and responsive, which led me to pursue more advanced real-time computer vision projects.	

Hand Tracking Computer Vision Guitar

Python • OpenCV • MediaPipe

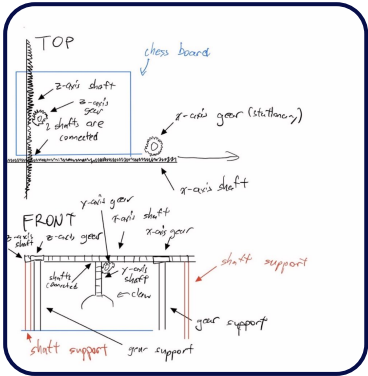


What? I worked on a team project to design and build an autonomous checkers-playing robot that could compete against a human player. The system was designed to scan the board, identify piece locations, determine a legal move, and physically execute that move using a motorized claw and gantry mechanism. Using VEX components and sensors, the robot tracked its position across the board, detected different piece types, and updated the game state in real time. The project combined mechanical design, embedded programming, and sensor integration to create a working mechatronic game-playing system.	How? The robot used an X-Y motion system, a claw mechanism, and optical and distance sensors to scan the board and move pieces. The software stored the board as a 2D array and handled piece detection, legal move selection, promotions, and game-over logic. The team also iterated on both the robot and the game pieces, improving chassis stability, reducing jamming, refining claw and sensor alignment, and adjusting the checkers pieces to better support reliable detection and handling.
---	---

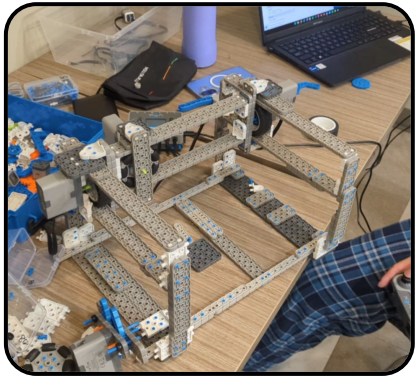
Impacts The final robot was able to scan the board, choose legal moves, and play a simplified game of checkers against a human. It demonstrated successful integration of mechanical design, sensors, and embedded software in one working robotic system. This project strengthened my skills in prototyping, embedded programming, debugging, and full-system mechatronic design.

Checkers Robot

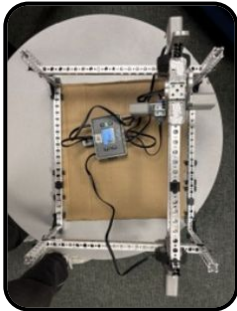
C++ • Robotics • Embedded Programming • VEX • Mechanical Design • Prototyping



Initial Concept



First Functional Prototype



Final Design



Demo

What? I developed Space Race, a 2D survival game in Java using JavaFX, where the player controls a spaceship, dodges randomly spawned asteroids, shoots aliens, and tries to survive as long as possible. The game includes a full menu flow, live gameplay, health tracking, enemy combat, and a game over screen with persistent high-score statistics.	How? I built the game using an object-oriented design with separate classes for the spaceship, aliens, asteroids, projectiles, health bar, background generation, and stat tracking, which kept gameplay systems modular and easier to manage. Real-time gameplay was driven through a JavaFX AnimationTimer loop that handled movement, spawning, collision detection, combat, and survival time updates. I also implemented smoother movement by tracking held keyboard inputs in a key-state map instead of relying only on default key press events. I programmed directional projectile motion by calculating normalized x and y velocity components so lasers could travel accurately toward mouse-click targets and enemy shots could track the player. I also added collision handling between the spaceship, asteroids, aliens, and projectiles, plus persistent stat saving for kills, survival time, and damage dealt using file I/O.
---	--

Impacts This project demonstrated my ability to build a complete interactive software application from concept to implementation. It also improved my ability to organize larger programs with multiple classes and build systems that work smoothly together in real time.

Space Race

Java • JavaFX • Game Development



What?

Made a spacer jig to help align push bar holes for accurate drilling.

How?

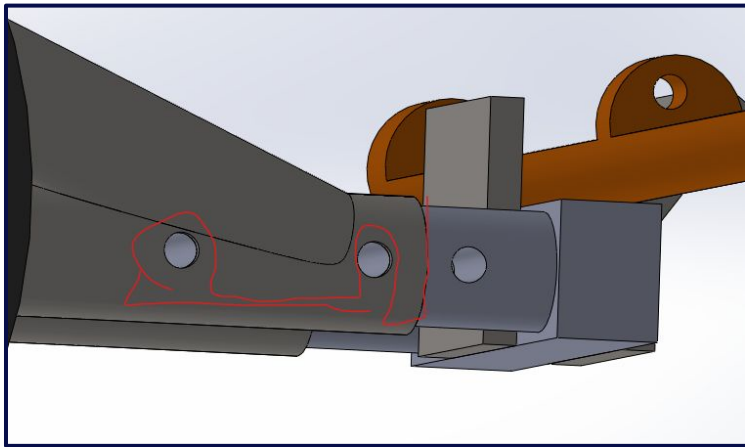
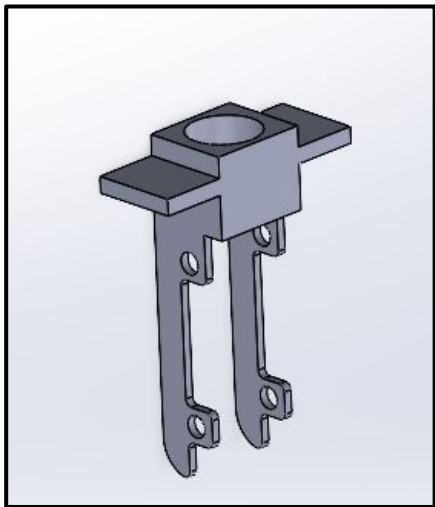
Measured the necessary dimensions on the driveline SVN assembly and modelled the spacer to fit the dimensions in SolidWorks and then 3D-Printed the jig

UWFE Push Bar Spacer

SolidWorks • 3D-Printing • SVN

Impacts

Team members were able to easily and accurately drill holes into the push bar with the jig.



What?

Designed and modeled a motor jig in SolidWorks to improve alignment and repeatability of chassis hole and insert manufacturing

Impacts

Enabled the driveline team members to perform fast and accurate machining by standardizing hole alignment.

How?

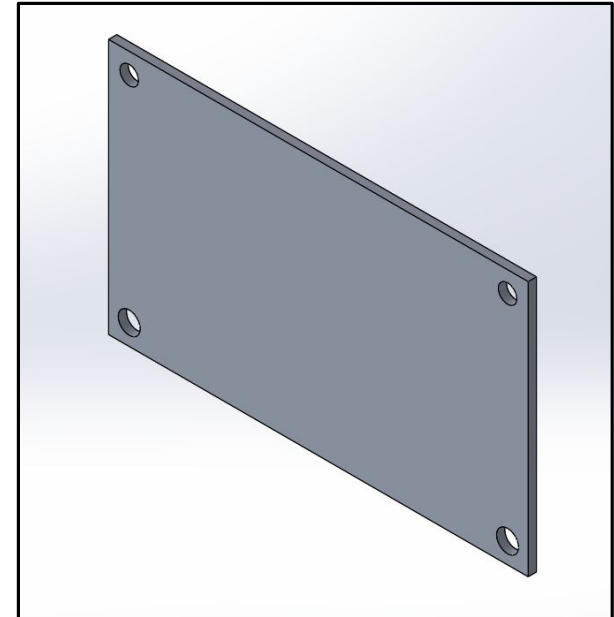
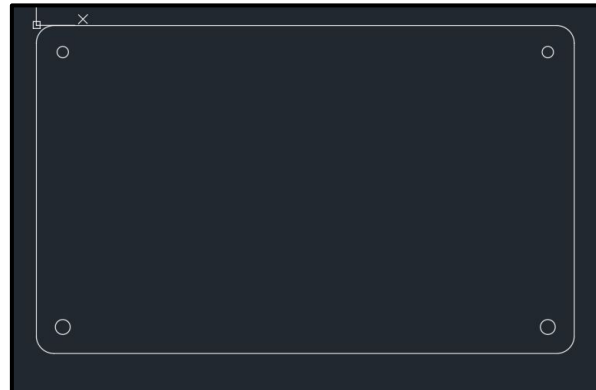
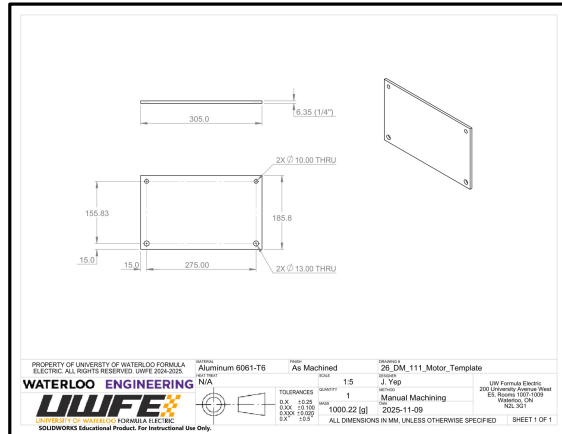
Designed and modelled the part using SolidWorks according to measurements taken from the driveline master assembly SVN.

Drafted an engineering drawing with GD&T specifications, and had it validated by a senior team member to ensure manufacturability and accuracy.

Water-jetted the base to specifications, and milled the holes to ± 0.1 mm tolerances.

UWFE Motor Template

SolidWorks • AutoCAD • SVN



What?

Accurately remodelled a radiator in CAD for the cooling system design.

How?

Measured the necessary dimensions on the radiator using a caliper then recreated the radiator in SolidWorks.

UWFE Radiator

SolidWorks • Caliper • SVN

Impacts

The CAD model was integrated into the driveline master assembly using TortoiseSVN, and was a key component in completing the cooling system design.

